Data types and structures

```
$ echo "Data Sciences Institute"
```

Overview

- Vectors (Wickham and Grolemund, 2017 Chapter 20)
- Tibbles (Wickham and Grolemund, 2017 Chapter 10)
- Strings (Wickham and Grolemund, 2017, Chapter 14)
- Factors (Wickham and Grolemund, 2017, Chapter 15)
- Dates and times (Wickham and Grolemund, 2017, Chapter 16)
- Missing values (Wickham and Grolemund, 2017 Chapter 5)

Atomic types in R

1. *Character* have quotes around them. `"welcome"`, `'hello world'`, and `"2"` are all of type character.

- May also be referred to as a string in some contexts

2. *Logical* is either `TRUE` or `FALSE`
3. *Double* is a number. `3.1`, `-73`, and `2700` are all doubles.
4. *Integer* ex. `100L`
5. Complex ex. 10+3i
6. Raw (byte representation)

You likely will only need to know the first four.

Atomic types in R

```
typeof("welcome")
#> [1] "character"
```

```
typeof(FALSE)
#> [1] "logical"
```

```
typeof(3.14)
#> [1] "double"
```

```
typeof(100L)
#> [1] "integer"
```

```
typeof(10+3i)
#> [1] "complex"
```

Vectors

Atomic vectors are made using the `c()` function.

We can build vectors of data out of all atomic data types. All the data types in an atomic vector need to match.

Lists, which are sometimes called recursive vectors, can contain other lists. They can also be heterogeneous, containing multiple types.

Logical Vectors

Possible values are `TRUE`, `FALSE`, and `NA`

Often created with comparison operators

```
logical_vector <- c(TRUE, TRUE, FALSE)
typeof(logical_vector)
length(logical_vector)

#which numbers between 1 and 5 are divisible by 2?
compare_vector <- 1:5 %% 2 == 0
typeof(compare_vector)
length(compare_vector)
compare_vector
```

Numeric Vectors

Integer and double vectors together are called numeric. Numbers in R are doubles by default, so you need to specify L to make an integer value.

```
double_vector <- c(3.1, -73, 2700)
typeof(double_vector)
length(double_vector)

integer_vector <- c(3L, -73L, 2700L)
typeof(integer_vector)
length(integer_vector)
```

Numeric Vectors

Differences:

1. Doubles are approximations, because floating point numbers cannot always be represented with a fixed amount of memory.
2. Special values:

- Integers have `NA`
- Doubles have `NA`, `NaN`, `Inf`, and `-Inf`

Numeric vectors

We can check for special values in general with `is.infinite`, `is.na`, and `is.nan`:

```
special_values <- c(-1, 0, 1, NA) / 0
special_values
#> [1] -Inf  NaN  Inf   NA

is.finite(special_values)
#> [1] FALSE FALSE FALSE FALSE

is.infinite(special_values)
#> [1]  TRUE FALSE  TRUE FALSE

is.na(special_values)
#> [1] FALSE  TRUE FALSE  TRUE

is.nan(special_values)
#> [1] FALSE  TRUE FALSE FALSE
```

Character Vectors

```
character_vector <- c("hello", "world", "2,000")
typeof(character_vector)
length(character_vector)
```

Augmented Vectors

Augmented vectors, which add metadata in the form of attributes to vectors, are the basis of many data types in R.

- Factors are made from integer vectors
- Dates and times are made from numeric vectors
- Data frames and tibbles are made from lists.

Coercion

You can coerce one type of vector to another explicitly:

```
character_vector <- c("1", "0", "1")
typeof(character_vector)

numeric_vector <- as.numeric(character_vector)
typeof(numeric_vector)

double_vector <- as.double(character_vector)
typeof(double_vector)

logical_vector <- as.logical(character_vector)
typeof(logical_vector)
```

Implicit Coercion

```r
numeric_vector <- 1:10

# which are greater than 4?
logical_vector <- numeric_vector > 4
logical_vector
# how many are greater than 4?
sum(logical_vector)
# what proportion are greater than 4?
mean(logical_vector)
```

Mixing Types

If you mix types in a vector, all types will be coerced to match the "most complex" type.

```
typeof(c(TRUE, FALSE, 10L))
typeof(c(1L, 4L, 1.5))
typeof(c(1.5, -3.2, "a"))
```

Checking type with tidyverse functions

```
is_logical(c(TRUE, FALSE))
is_integer(c(1L, 2L))
is_double(c(1.2, 1.3))
is_character(c("hello", "world"))
is_atomic(c(1,2,3))
is_list(c(list(1,2,3)))
is_vector(c(1,2,3))
```

Vector Recyling

If an operation requires a longer vector than provided, R will recycle the vector to get to the required length:

```
1:5 + 1:10
#>  [1]  2  4  6  8 10  7  9 11 13 15
```

It will also warn you if the recycled vector isn't a complete multiple of the smaller vector:

```
1:5 + 1:13
#> Warning in 1:5 + 1:13: longer object length is not a multiple of shorter object length
#>  [1]  2  4  6  8 10  7  9 11 13 15 12 14 16
```

Naming Vectors

```
named_vector <- c(a = 100, b = 90, c = 80, d = 70, e = 60)
named_vector
#>   a   b   c   d   e
#> 100  90  80  70  60
```

Named vectors are good if you want to subset.

Subsetting

You can subset with a numeric vector containing only integers:

```
named_vector[3]
#>  c
#> 80

named_vector[c(3,3,4)]
#>  c  c  d
#> 80 80 70

named_vector[c(-1,-2,-5)]
#>  c  d
#> 80 70
```

Subsetting

You can subset with a logical vector:

```
named_vector[c(TRUE, TRUE, FALSE, TRUE, FALSE)]
#>   a   b   d
#> 100  90  70
```

```
named_vector[named_vector %% 20 == 0]
#>   a   c   e
#> 100  80  60
```

Subsetting

You can subset with a character vector:

```
named_vector[c("a", "c")]
#>   a   c
#> 100  80
```

Lists

Because a list can contain other lists, they can represent hierarchical structures.

```
mylist <- list(7, "abc", FALSE)
mylist
#> [[1]]
#> [1] 7
#>
#> [[2]]
#> [1] "abc"
#>
#> [[3]]
#> [1] FALSE
```

Subsetting lists

```
mylist <- list(a = 1:4, b = "zyx", c = list(-1, -5))

mylist[1:2]
```

Extracting items

```
mylist[[2]]

mylist[["b"]]

mylist$b
```

Additional Attributes of Objects

- Names
- Dimensions (vector behaves like a matrix or array)
- Class

Tibbles

R has data.frames for storing columns and rows of data, but in tidyverse we have tibbles instead.

Tibbles are augmented lists. All elements of the tibble must be vectors with the same length. The same applies to data.frames.

You can create a new tibble as follows:

```
mytibble <- tibble(x = 1:5,
        y = 1,
        z = x ^ 2 + y)
mytibble
```

Coercing between data frames and tibbles

```
library(palmerpenguins)
data(palmerpenguins)
head(penguins)
```

```
penguins_df <- as.data.frame(penguins)
head(penguins_df)
```

```
penguins_tibble_again <- as_tibble(penguins_df)
head(penguins_tibble_again)
```

Some differences between data.frames and tibbles

- Tibbles are special cases of data.frames
- Tibbles print more nicely and are easier to read in the console
- Subsetting works differently in some cases
- Tibbles are "stricter" and so they're safer to use
    - more predictable behaviour

For most purposes, data.frames and tibbles are interchangeable.

Subsetting data.frames

```
penguins$species
```

```
penguins[["species"]]
```

Subtle differences between tibbles and data.frames

data.frames allow for "partial" string matching for column names. Compare:

```
penguins$sp
```

```
penguins_df$sp
```

Subtle differences between tibbles and data.frames

Accessing columns with `[]` syntax returns different types of objects. Compare:

```
penguins["species"]
```

```
penguins_df["species"]
```

Strings

```
library(stringr) # part of the tidyverse
```

Strings are contained between single '' or double "" quotes.

```
"This is a string"
'6' # this is ALSO a string
```

Check the length

```
str_length("This is a string")
#> [1] 16
```

Strings

Combine

```
str_c("This is a string", "6")
#> [1] "This is a string6"
```

Take a subset

```
library(stringr)
str_sub("This is a string", 7, 12)
#> [1] "s a st"
```

Strings

Change capitalization

```
str_to_lower("UPPER case")
#> [1] "upper case"

str_to_upper("LOWER case")
#> [1] "LOWER CASE"

str_to_title("no capitalization")
#> [1] "No Capitalization"
```

Matching patterns

```
mystring <- c("apple", "banana", "clementime", "dragonfruit")

str_view(mystring, "an")
#> [2] │ b<an><an>a
```

Regular expressions

A period matches any character.

```
mystring <- c("apple", "banana", "clementime", "dragonfruit")
str_view(mystring, ".a.")
```

If you want to actually match a period, you need to use a double backslash. `\.` is an escape character for the period, and `\\.` an additional escape symbol for the `\`

```
"\\."
```

And if you want to actually match a backslash, you need to use a quadruple backslash:

```
"\\\\"
```

Regex Anchors

`^` matches to the start of a string

```
str_view(mystring, "^a")
```

`$` matches to the end of a string

```
str_view(mystring, "a$")
```

Classes

- `\d` matches any digit. (Remember that it will need an additional escape character.)
- `\s` matches any whitespace. (Remember that it will need an additional escape character.)
- `[xyz]` matches x, y, or z
- `[^xyz]` matches anything except x, y, or z

Amounts

- `?` matches 0 or 1
- `+` matches 1 or more
- `*` matches 0 or more

```
mystring <- "abcccdeee"

str_view(mystring, "cc?")
str_view(mystring, "cc+")
str_view(mystring, "c[de]+")
```

Specifying exact number of matches

- `{n}` matches exactly n
- `{n,}` matches n or more
- `{,m}` matches m or less
- `{n,m}` matches at least n and at most m

Disambiguating

We can use parentheses in complex expressions to make multiple requirements. For example, finding repeated pairs:

```
mystring <- c("abab", "cdcd", "efgh")
str_view(mystring, "(..)\\1", match = TRUE)
```

Using regex

```
mystring <- c("banana", "dodo", "apple")

str_detect(mystring, "(..)\\1")
str_subset(mystring, "(..)\\1")
str_count(mystring, "(..)\\1")
```

Factors

In R, factors are for working with categorical variables, where there is a fixed and known set of possible values.

```
library(forcats) # part of the tidyverse
```

Let's say we have a variable storing the months of our data.

```
months <- c("Dec", "Apr", "Jan", "Mar")
months
```

Factors

With a factor, you can restrict the number of possible values and order those values.

```
month_levels <- c(
  "Jan", "Feb", "Mar", "Apr", "May", "Jun",
  "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"
)
```

```
month_fix <- factor(months, month_levels)
month_fix
```

Recoding factors

If we wanted all the levels to be full month names instead, we could recode the levels:

```
fct_recode(month_fix, "December" = "Dec")
```

Dates

- Dates in R are numeric vectors that represent number of days since January 1, 1970.
- Tibbles print this as <date>.

```
today()
```

Time

- time within a day: tibbles print this as .

Datetime

- date-time: a date plus a time that uniquely identifies an instant in time.
- Numeric vectors that represent the number of seconds since January 1, 1970.
- Tibbles print this as <dttm>.
- Elsewhere in R these are called POSIXct.

```
now()
```

Managing dates using tidyverse

You will primarily use the library `lubridate`, and not see POSIXcts very frequently.

```
library(lubridate)

lubridate::as_datetime(<POSIXct item>)
```

Parsing dates from strings and numbers

```
ymd("2017-01-31")
ymd(20170131) # the most concise way
mdy("January 31st, 2017")
dmy("31-Jan-2017")
```

Switching between date and datetime

```
today()
as_datetime(today())

now()
as_date(now())
```

Components

- We can extract year, month, day of the month, day of the year, day of the week, hour, minute, and second:

```
datetime <- ymd_hms("2016-07-08 12:34:56")

year(datetime)
month(datetime)
mday(datetime)
yday(datetime)
wday(datetime)

hour(datetime)
minute(datetime)
second(datetime)
```

Time spans

```
today() - ymd(20000101)
as.duration(today() - ymd(20000101))
```

```
dseconds(120)
dminutes(60)
dhours(c(12, 24))
ddays(0:7)
dweeks(4)
dyears(10)
```

Periods

Periods are time spans that don't have fixed length in seconds, so they work more like you might anticipate.

```
today() + years(1)
today() + months(1)
today() + days(1)
today() + hours(1)
today() + minutes(1)
today() + seconds(1)
```

Time zones

```
ymd_hms("2021-01-01 12:00:00", tz = "America/New_York")
ymd_hms("2021-01-01 18:00:00", tz = "Europe/Copenhagen")
ymd_hms("2021-01-01 04:00:00", tz = "Pacific/Auckland")
```

Missing data

Comparisons do not work as expected with missing values.

```
NA > 5
NA == 10
NA + 5
NA == NA
```

Detect missing values with:

```
is.na(NA)
```

Exercises

Exercises

1. Make a tibble where the vectors do not have equal length. What happens?
2. In the following tibble, extract variable:

```
mytibble <- tibble(
  A = 1:10,
  B = A * 2)
```

3. Try using functions `paste()` and `paste0()`. Compare them to `str_c`. How do they work differently?
4. Look up function `str_trim()` and demonstrate application.
5. Match the sequence "'\ with regex
6. Match words that start with x with regex.

Exercises

7. Match words that are 3 letters long with regex.
8. Match words that only contain consonants with regex.
9. What does ^.*$ match?
10. What happens if you parse a string with invalid dates?

Any questions?